

Surfcast - Project Report

Purpose

Surfcast is a web application that collects surf conditions for several surf breaks on Oahu, gives each break a surf-quality score, and ranks the breaks against each other. The problem it solves is a small but real one: there are good forecast sites out there, but they show raw numbers for one spot at a time, so a surfer still has to open several of them and guess which break is actually best right now. Surfcast does that comparison and presents a single ranked list.

Target audience

The product is aimed at surfers on Oahu who are deciding where to surf on a given day. Oahu is a good fit because its breaks face different directions, so the best spot changes a lot with the season and the conditions - in summer the south shore (Waikiki) works while the famous North Shore breaks are flat, and in winter it is the other way round. That is exactly the kind of "where should I go today" decision the ranking is meant to help with.

What makes it unique

Most weather and surf pages are a dump of raw forecast data for one location. Surfcast is different in that it turns the raw data into one comparable score per break and ranks the breaks against each other. The user gets an answer ("Waikiki is best right now"), not just a table of numbers to interpret themselves.

Requirements

The full list is in `docs/requirements.md`. In short, the user can see the ranked breaks, see the conditions behind each score, ask for a break's average score over a date range, and read the data as JSON. The system collects conditions for several breaks from a public API, stores them, scores them, serves a web page and a REST API, runs in production, is covered by tests in CI, can collect through a message queue, and exposes monitoring endpoints.

Design decisions and justification

Tech stack

- **Python and Flask** for the web app. Flask is small and was the framework used in the course, so it kept the app simple to build and to deploy.
- **SQL storage with Flask-SQLAlchemy**. The readings are uniform, table-shaped rows (a break name, a time, and a few numbers), which is exactly what a SQL database is good at, so a relational store fit better than a NoSQL one. Locally the app uses SQLite because it needs no separate server. In production it uses a hosted Postgres database instead, because Render's free tier wipes the local disk on restart so SQLite data would not survive there. The database location is read from the `SURFCAST_DATABASE_URI` environment variable, so the same code runs against either one with no changes.

- **Open-Meteo** as the data source. It is free, needs no API key, and has both a normal forecast API (temperature and wind) and a marine API (wave height and period), so it covers everything the score needs from one provider.
- **Render** for hosting. It gives a free public URL and redeploys automatically when I push to the main branch.
- **GitHub Actions** for continuous integration and **RabbitMQ (CloudAMQP)** with the **Pika** client for the message queue. These are explained below.

Architecture



The collector, the consumer and the web app all share one database model (`src/models.py`) so they read and write the same data.

Computing the score in the analyzer instead of storing it

The surf-quality score is not stored in the database. The collector only stores the raw conditions, and the analyzer works out the score when the data is read. This keeps the collector simple and, more importantly, lets the scoring be tested on its own and changed later without having to re-collect any data.

The surf-quality score

The score is a 0-100 heuristic. Bigger waves, longer period and lighter wind all make for better surf, so the score adds up three parts:

- wave height, up to 40 points (caps out at 3 m),
- wave period, up to 30 points (caps out at 14 s),
- wind, up to 30 points (full points for no wind, zero at 40 km/h or more).

It is a rough model rather than a scientific one, but it captures the main idea that waves matter most, then period, then wind.

Why a message queue

For the highest level of the rubric the project needed event-collaboration messaging. A message queue is also a genuinely reasonable design here: collecting the data and storing it are two separate jobs, and putting a queue between them means the collector can just publish readings and not care about the database, while one or more consumers do the storing. The collector publishes to a RabbitMQ queue when `RABBITMQ_URL` is set, and `consumer.py` reads the queue and saves the readings. When no broker is configured the collector just writes to the database directly, so the app still runs without RabbitMQ.

Testing strategy

- **Unit tests** (`tests/test_analyzer.py`) check the scoring and ranking with fixed inputs and known answers.
- **Mock-object tests** (`tests/test_collector.py`, `tests/test_messaging.py`) replace the Open-Meteo API and the RabbitMQ broker with mocks. This means the tests do not depend on the network, on live surf conditions, or on a running broker, so they give the same result every time.
- **Integration tests** (`tests/test_integration.py`) drive the real web app, the database and the analyzer together through Flask's test client against a throwaway database.

The tests use a separate temporary database so they never touch the real data.

Continuous integration and delivery

Every push runs the tests on GitHub Actions (`.github/workflows/ci.yml`), which installs the dependencies and runs the whole test suite. Delivery is continuous too: Render is connected to the main branch and redeploys the live site automatically whenever main is updated.

The data collection also runs on a schedule with GitHub Actions (`.github/workflows/collect.yml`, every three hours). That workflow runs the collector against the production Postgres database using a `SURFCAST_DATABASE_URI` secret, which keeps the live site's data fresh without a paid scheduler.

Monitoring

The app logs every request and has two endpoints for monitoring: `/health`, which returns OK and also checks that the database is reachable (an uptime monitor can ping it), and `/stats`, which reports how many readings have been collected, the count per break, and when the last reading was stored.

System requirements to run

- Python 3.14.
- The packages in `requirements.txt` (Flask, gunicorn, Flask-SQLAlchemy, requests, pika, psycopg2-binary).
- Internet access for the collector to reach the Open-Meteo API.

- For production data: a hosted Postgres database, with its connection string in the `SURFCAST_DATABASE_URI` environment variable on both the Render web service and the GitHub Actions collector (as a repository secret). Without it the app falls back to local SQLite.
- For the message-queue path: a RabbitMQ broker (a free CloudAMQP instance works) with its connection string in the `RABBITMQ_URL` environment variable.
- For hosting: a Render web service running `gunicorn src.app:app`.

How the project meets the rubric

Rubric item	Where it is
Web application (form + reporting)	<code>src/app.py</code> - ranked table and average form
Data collection	<code>src/collect_weather.py</code> - Open-Meteo forecast and marine APIs
Data analyzer	<code>src/analyzer.py</code> - score, ranking, averages
Unit tests	<code>tests/test_analyzer.py</code>
Data persistence	SQLite via Flask-SQLAlchemy (<code>src/models.py</code>)
REST API endpoint	<code>/api/breaks</code> and <code>/api/breaks/<name>/average/<start>/<end></code>
Production environment	Render, public URL
Integration tests	<code>tests/test_integration.py</code>
Mock objects	<code>tests/test_collector.py</code> , <code>tests/test_messaging.py</code>
Continuous integration	<code>.github/workflows/ci.yml</code>
Production monitoring	<code>/health</code> , <code>/stats</code> , request logging
Event collaboration messaging	RabbitMQ producer/consumer (<code>src/messaging.py</code> , <code>src/consumer.py</code>)
Continuous delivery	Render auto-deploys the main branch

Limitations and future work

- The score is a simple heuristic, not a validated surf model. It could be tuned with real surfer feedback or take wind direction (offshore vs onshore) into account per break.
- The free Render web service sleeps when idle, so the first visit after a while is slow while the instance wakes up. A paid instance would stay awake.
- Only a handful of breaks and a few conditions are tracked. Tide and swell direction would make the score more accurate, and more breaks could be added.